



数字图像处理

Digital Image Processing

第10章 图像分割

Image Segmentation

PART B—阈值法、区域法分割

张朝晖
河北师范大学
2024.9~2024.11

主要内容

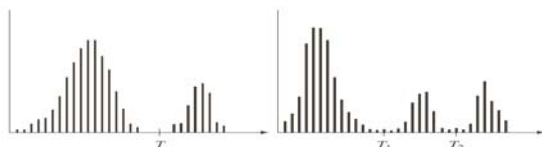
- 10.1 基础知识
- 10.2 点、线和边缘检测
- 10.3 阈值处理
 - 阈值法分割基础
 - 阈值法分类
 - 几种典型阈值分割法

10.4 基于区域的分割

2

考察基于图像灰度值/灰度值特性，将图像划分为若干区域。——阈值法图像分割

例：某输入图像 f 的灰度直方图

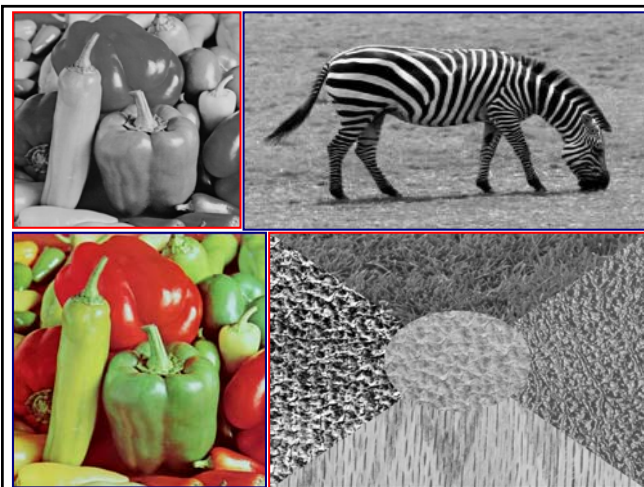


基于阈值 T 的图像分割：

$$g(x,y) = \begin{cases} a & \text{若 } f(x,y) > T \\ b & \text{若 } f(x,y) \leq T \end{cases}$$

基于阈值 T_1, T_2 的图像分割：

$$g(x,y) = \begin{cases} a & \text{若 } f(x,y) > T_2 \\ b & \text{若 } T_1 < f(x,y) \leq T_2 \\ c & \text{若 } f(x,y) \leq T_1 \end{cases}$$



阈值法分割的关键：

- 确定按照图像的何种特征进行阈值分割
- 阈值估计

$$\text{阈值 } T = T(x, y, f(x, y), p(x, y))$$

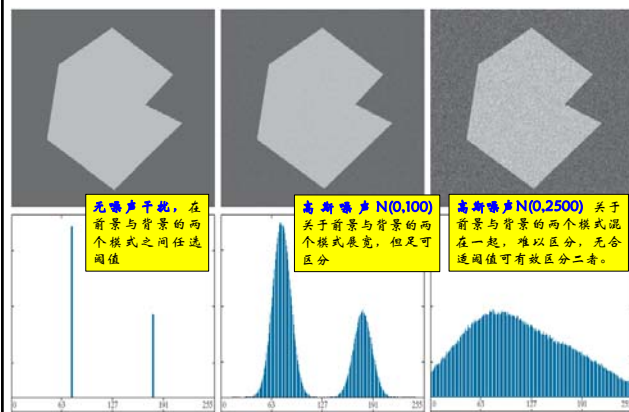
全局阈值化-- T 取决于 $f(x, y)$

局部阈值化-- T 取决于 $\begin{cases} \text{灰度值 } f(x, y) \\ \text{局部特征 } p(x, y) \end{cases}$

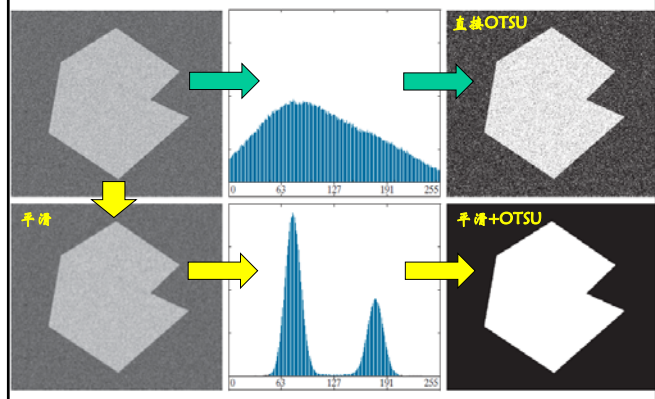
动态或自适应阈值化-- T 取决于位置 x, y

5

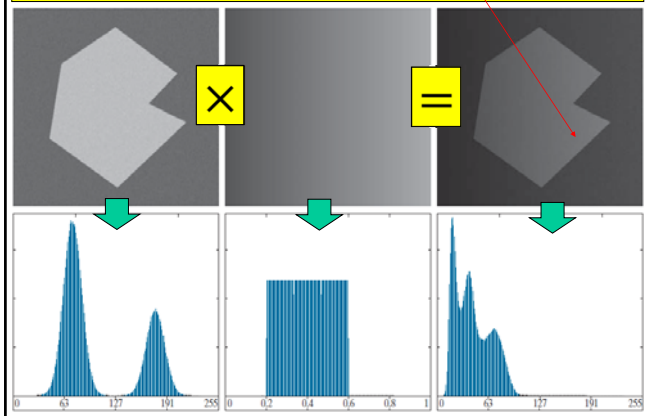
A. 基于灰度直方图进行阈值推断，阈值估计是否有效，受图像的噪声内容影响。



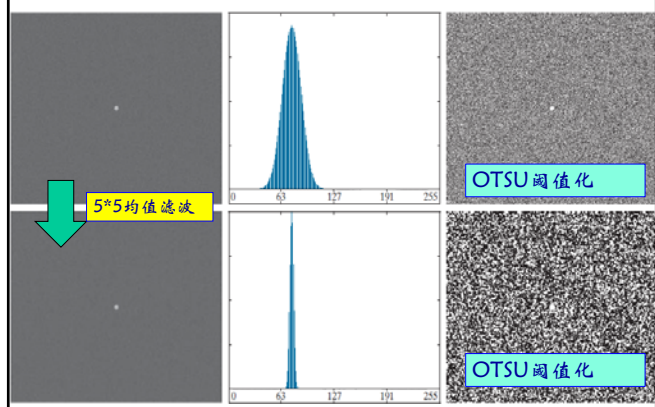
处理策略：图像平滑，抑制噪声，达到改善阈值效果的作用



B. 基于灰度直方图进行阈值推断，是否有效，与光源的均匀性、图像反射特性的均匀性有关。



C. 基于灰度直方图进行阈值推断，是否有效，与物体和背景的相对尺寸有关。



1. 基本全局阈值处理—迭代自适应单阈值估计 (page478)

设：分割前——灰度图像 $f(x,y)$, 有效灰度范围 $[l_1, l_2]$

分割后——二值输出图像 $g(x,y)$

预设阈值参数 ΔT

待估计灰度全局阈值 T

STEP1: 全局阈值 T 的初始化 $T^{(0)}$.

计算输入图像的归一化灰度直方图 $h(i), i = l_1, \dots, l_2$
以图像灰度均值，或 $\frac{l_1 + l_2}{2}$ 初始化全局阈值 $T^{(0)}$

STEP2: 全局阈值 T 的迭代估计.

A. 阈值 $T^{(k)}$ 将输入图像各像素分成两个集合

集合 $G_1^{(k)} = \{(x,y) | T^{(k)} < f(x,y) \leq l_2\}$
集合 $G_2^{(k)} = \{(x,y) | l_1 \leq f(x,y) \leq T^{(k)}\}$

10

STEP2: 全局阈值的迭代估计.

B. 估计两个集合的灰度均值:

$$m_1^{(k)} = \frac{\sum_{i=T^{(k)}+1}^{l_2} i \cdot h(i)}{\sum_{i=T^{(k)}+1}^{l_2} h(i)} \quad m_2^{(k)} = \frac{\sum_{i=l_1}^{T^{(k)}} i \cdot h(i)}{\sum_{i=l_1}^{T^{(k)}} h(i)}$$

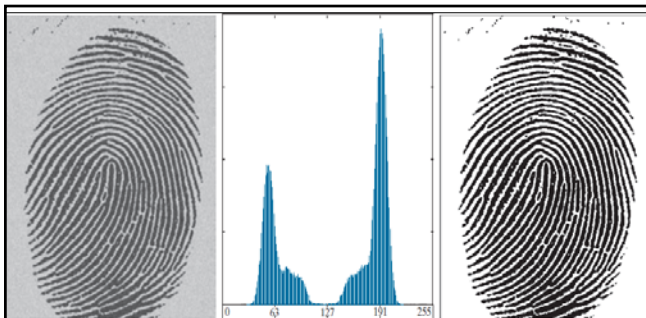
C. 第 $k+1$ 次迭代的全局阈值 $T^{(k+1)} = \frac{m_1^{(k)} + m_2^{(k)}}{2}$

重复 A, B, C, 直到 $|T^{(k)} - T^{(k+1)}| < \Delta T$, 迭代终止, 得全局阈值 $T = T^{(k+1)}$

STEP3: 输出分割图像 g .

$$g(x,y) = \begin{cases} a & \text{若 } f(x,y) \leq T \\ b & \text{若 } f(x,y) > T \end{cases}$$

11



含噪声成分的指纹图像自动全局阈值化处理:

左: 输入图像

中: 灰度直方图

右: 自动阈值化的二值图像 ($T=125$)

12

2. 基于Otsu法的最佳全局阈值处理—类间方差最大法

出发点:

➤ 将图像分割视为一种**两类(单阈值估计)**或**多类(多阈值估计)**情况下, 基于灰度特征的**最优分类问题**。

➤ 基于估计阈值得到的两类, **总的类间灰度方差**尽可能大。

➔ **准则函数: 最大类间灰度方差**

以**单阈值估计**为例, 设灰度图像由C1、C2两类组成。

13

设 $\begin{cases} \text{分割前--灰度图像 } f, \text{有效灰度范围 } [0, L-1] \\ \text{分割后--输出图像 } g \\ \text{待估计灰度全局阈值 } T \end{cases}$

选择一个**阈值** $T(k) = k$, $0 \leq k < L-1$

该**阈值** k 将输入图像各像素分为两类 C_1, C_2

$\begin{cases} \text{低灰度区 } C_1 \text{类} = \{(x,y) | 0 \leq f(x,y) \leq T(k)\} \\ \text{高灰度区 } C_2 \text{类} = \{(x,y) | T(k)+1 \leq f(x,y) \leq L-1\} \end{cases}$

14

输入图像 f 对应 **灰度值** i 的像素数目 n_i

输入图像 f 中 **灰度值** i 出现的概率 $p_i = \frac{n_i}{\sum_{j=0}^{L-1} n_j}$

C_1 类, C_2 类**先验概率**: $P_1 = \sum_{i=0}^k p_i$ $P_2 = \sum_{i=k+1}^{L-1} p_i = 1 - P_1$

C_1 类像素灰度均值 $m_1 = \frac{\sum_{i=0}^k i n_i}{\sum_{i=0}^k n_i} = \frac{\sum_{i=0}^k i n_i}{\sum_{i=0}^{L-1} n_i} \cdot \frac{\sum_{i=0}^{L-1} n_i}{\sum_{i=0}^k n_i} = \frac{\sum_{i=0}^k i p_i}{\sum_{i=0}^k p_i} = \frac{\sum_{i=0}^k i p_i}{P_1}$

C_2 类像素灰度均值 $m_2 = \frac{\sum_{i=k+1}^{L-1} i p_i}{\sum_{i=k+1}^{L-1} p_i} = \frac{\sum_{i=k+1}^{L-1} i p_i}{P_2}$

图像 f 的全局灰度平均值 $m_G = \sum_{i=0}^{L-1} i p_i = \sum_{i=0}^k i p_i + \sum_{i=k+1}^{L-1} i p_i = P_1 m_1 + P_2 m_2$

阈值 $T(k) = k$ 将**输入图像**分为两类 C_1, C_2 , 导致的**类间灰度方差**:

$$\begin{aligned} \sigma_{\text{Between}}^2(k) &= P_1(k) [m_1(k) - m_G]^2 + P_2(k) [m_2(k) - m_G]^2 \\ &= P_1(k) \left[\overline{m_1(k) - P_1(k) m_1(k)} - P_2(k) m_2(k) \right]^2 \\ &\quad + P_2(k) \left[m_2(k) - P_1(k) m_1(k) - P_2(k) m_2(k) \right]^2 \\ &= P_1(k) \left[P_2(k) m_1(k) - P_2(k) m_2(k) \right]^2 \\ &\quad + P_2(k) \left[P_1(k) m_2(k) - P_1(k) m_1(k) \right]^2 \\ &= P_1(k) P_2^2(k) [m_1(k) - m_2(k)]^2 + P_2(k) P_1^2(k) [m_2(k) - m_1(k)]^2 \\ &= P_1(k) P_2(k) [m_1(k) - m_2(k)]^2 [P_1(k) + P_2(k)] \\ &= \overline{P_1(k) P_2(k)} [m_1(k) - m_2(k)]^2 \end{aligned}$$

16

好的阈值划分结果应该 $\sigma_{\text{Between}}^2$ 尽量大

$$\sigma_{\text{Between}}^2(k) = P_1(k) [m_1(k) - m_G]^2 + P_2(k) [m_2(k) - m_G]^2 = P_1(k) P_2(k) [m_1(k) - m_2(k)]^2$$

最优阈值 $T = k^* = \arg \max_{0 \leq k < L-1} \sigma_{\text{Between}}^2(k)$

$$\text{归一化测度: } \eta(k) = \frac{\sigma_{\text{Between}}^2(k)}{\sum_{i=0}^{L-1} (i - m_G)^2 p_i}$$

分割后的输出图像 g : $g(x,y) = \begin{cases} a(\text{如}0) & \text{若 } f(x,y) \leq T \\ b(\text{如}255) & \text{若 } f(x,y) > T \end{cases}$

17

Otsu法算法步骤小结:

1. 计算输入图像的归一化灰度直方图 $\{p_i, i=0, \dots, L-1\}$

2. 计算输入图像全局灰度均值 $m_G = \sum_{i=0}^{L-1} i p_i$

3. for $k = 0, \dots, L-2$

3.1 计算累积和 $P_1(k) = \sum_{i=0}^k p_i$

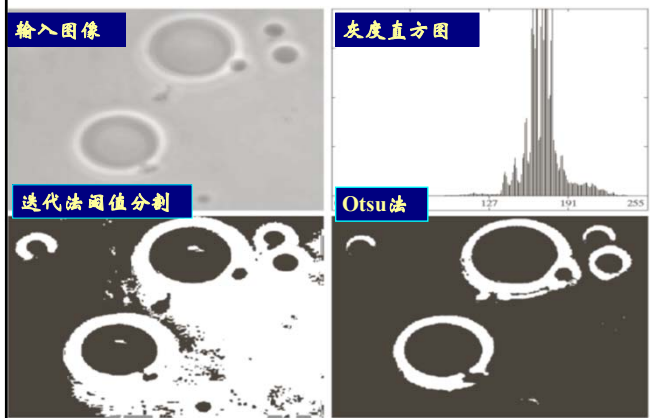
3.2 计算累积均值 $m_1(k) = \frac{\sum_{i=0}^k i p_i}{\sum_{i=0}^k p_i} = \frac{\sum_{i=0}^k i p_i}{P_1(k)}$

3.3 计算: $P_2(k) = 1 - P_1(k)$ $m_2(k) = \frac{m_G - P_1(k) m_1(k)}{1 - P_1(k)}$

3.4 计算类间方差 $\sigma_{\text{Between}}^2(k)$: $\sigma_{\text{Between}}^2(k) = P_1(k) P_2(k) [m_1(k) - m_2(k)]^2$

4. 确定最优阈值 $\begin{cases} \text{若 } \sigma_{\text{Between}}^2 \text{最大值唯一, 则 } k^* = \arg \max_{0 \leq k < L-1} \sigma_{\text{Between}}^2(k) \\ \text{否则, 取所有 } \sigma_{\text{Between}}^2 \text{最大值对应 } k \text{的平均值为 } k^* \end{cases}$

例：Otsu法的最佳全局阈值分割示例



3. 以背景为主的小目标区域分割—基于图像的边缘特性改进全局阈值处理(page 484)

基本思想：

- 如果直方图的各个波峰很高、很窄、对称，且被很深的波谷分开时，有利于选择阈值
- 为改善直方图的波峰形状，直方图的构建只利用**位于或靠近图像区域边缘的像素**，而不考虑目标区域中间的像素。
- 这些边缘像素位于目标区域的概率与背景的概率几乎一致，有助于得到比较对称的直方图模式。
- 可借助梯度幅值阈值化方式、或拉普拉斯微分算子，处理图像，选取图像边缘附近的像素

20

基于图像边缘特性的直方图形状改善，优点：

- 当目标前景和背景所占区域面积差别很大时，不会造成一个灰度级的波峰过高，而另一个过低
- 位于图像边缘上的像素在区域内还是区域外的概率是相等的，使用靠近边缘附近的像素生成灰度直方图可以增加波峰的对称性
- 基于梯度和拉普拉斯算子辅助选择像素，可以增加直方图的波峰高度

21

基于图像边缘特性，改善全局阈值选择的方法实现：

STEP1. 计算输入图像 f 在每个像素处的梯度，得到**梯度幅值图像**

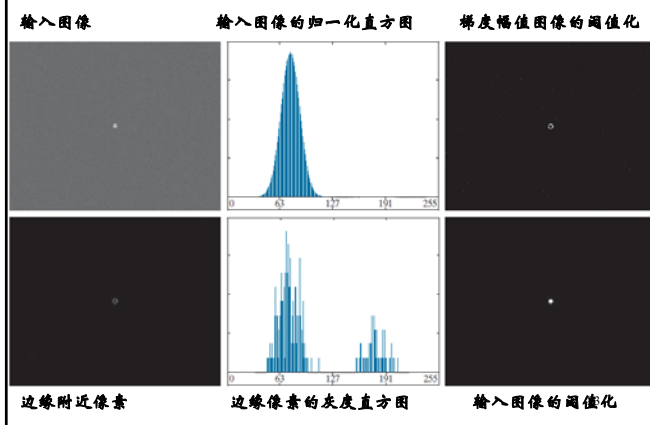
STEP2. 设定合适的阈值，确定梯度幅值取值足够大的像素(例如前10%)，得到这部分像素的归一化灰度直方图

注意：也可采用拉普拉斯算子，过滤图像，找到过零点处像素，以过零点处像素的灰度均值为阈值 T

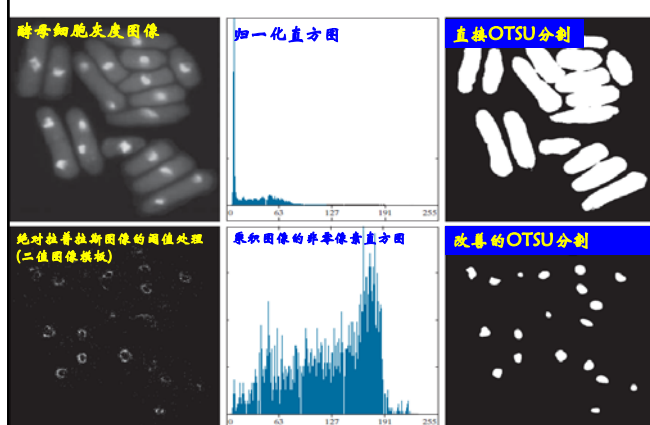
STEP3. 通过直方图的谷底，得到阈值 T

22

例：基于图像的梯度信息，获取边缘附近像素，改善全局阈值



例：基于图像拉普拉斯运算，获取边缘附近像素，改善全局阈值



4. 基于Otsu法的图像多阈值处理——类间方差最大法(page 547)

出发点:

- 将图像分割视为一种多类(多阈值估计)情况下, 基于灰度特征的最优分类问题。
- 图像多阈值分割后, 各类总类间灰度方差尽可能大。

→ 准则函数: 最大类间灰度方差

基于K-1个阈值, 将灰度图像划分成K个像素集合(K类)。

25

设 { 分割前--灰度图像 $f(x,y)$, 有效灰度范围 $[0, L-1]$
分割后--输出图像 $g(x,y)$
 $K-1$ 个待估计灰度阈值 $T_1^*, \dots, T_{K-1}^*$

$$0 \leq T_1^* < \dots < T_{K-1}^* < L-1$$

该阈值 $T_1, \dots, T_{K-1}$ 将输入图像分为 K 类 $C_1, \dots, C_K$

$$\begin{cases} C_1 \text{ 类} = \{(x,y) | 0 \leq f(x,y) \leq T_1\} \\ C_2 \text{ 类} = \{(x,y) | T_1 + 1 \leq f(x,y) \leq T_2\} \\ \dots \\ C_K \text{ 类} = \{(x,y) | T_{K-1} + 1 \leq f(x,y) \leq L-1\} \end{cases}$$

26

输入图像 f 中对应灰度值 i 的像素数目 n_i

输入图像 f 归一化灰度直方图:

$$\text{灰度值出现的概率 } p_i = \frac{n_i}{\sum_{j=0}^{L-1} n_j}$$

$$C_k \text{ 类先验概率: } P_k = \sum_{i=T_{k-1}+1}^{T_k} p_i$$

$$C_k \text{ 类像素灰度均值 } m_k = \frac{\sum_{i=T_{k-1}+1}^{T_k} i p_i}{P_k}$$

$$\text{图像 } f \text{ 的全局灰度平均值 } m_G = \sum_{i=0}^{L-1} i p_i = \sum_{k=1}^K P_k m_k$$

27

阈值 $T_1, \dots, T_{K-1}$ 将输入图像分为 K 类 $C_1, \dots, C_K$, 总类间灰度方差:

$$\sigma_{\text{Between}}^2(T_1, \dots, T_{K-1}) = \sum_{k=1}^K P_k (T_1, \dots, T_{K-1}) [m_k(T_1, \dots, T_{K-1}) - m_G]^2$$

最优阈值 $T_1^*, \dots, T_{K-1}^*$ 应满足:

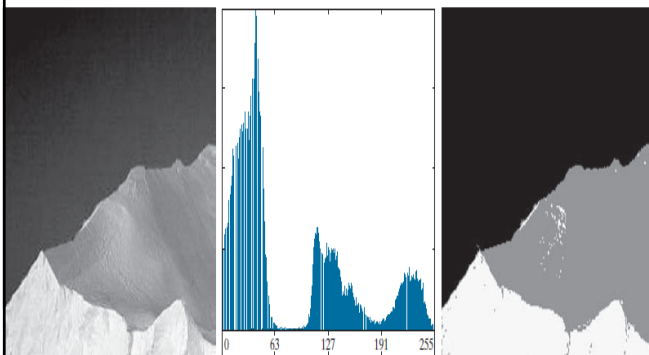
$$[T_1^*, \dots, T_{K-1}^*] = \arg \max_{0 \leq T_1 < \dots < T_{K-1} < L-1} \sigma_{\text{Between}}^2(T_1, \dots, T_{K-1})$$

$$\text{分割后的输出图像 } g: \quad g(x,y) = \begin{cases} I_1 & \text{若 } f(x,y) \leq T_1^* \\ I_2 & \text{若 } T_1^* + 1 \leq f(x,y) \leq T_2^* \\ \dots & \dots \\ I_K & \text{若 } T_{K-1}^* + 1 \leq f(x,y) \leq L-1 \end{cases}$$

28

例: 基于双阈值估计的冰山图像分割。

输出--三值图像: 暗背景、冰山明亮区域、冰山阴影区域



5. 可变阈值处理

方式1--基于图像分块的可变阈值处理

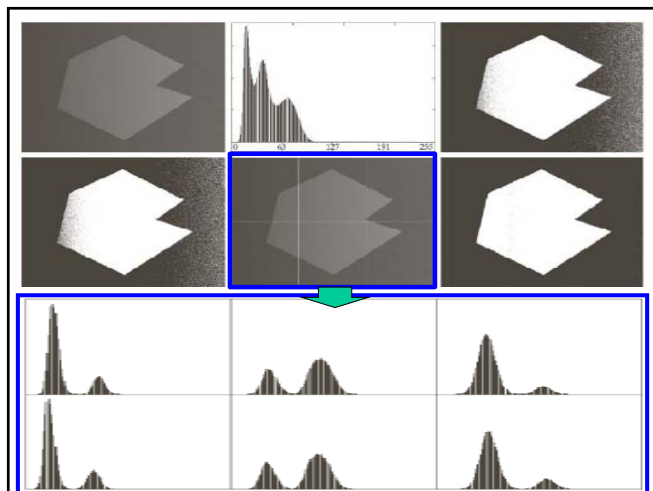
(1) 前提

- 感兴趣的目标区域和背景空间占比合理
- 图像中存在光照或反射的不均匀性

(2) 处理方式

- 将输入图像划分成若干互不重叠的矩形, 确保矩形区块足够小, 使得每个区块的光照近似均匀
- 分别针对每个矩形区块, 估计阈值、阈值化处理
- 将各区块阈值化结果合并, 得到输出的阈值图像

30



方式2—基于图像局部统计特性的可变阈值处理

处理方式

➤ 遍历输入图像，获取输入图像的全局灰度统计特性，如：灰度均值

对于图像的以像素坐标 (x, y) 为中心的邻域 S_{xy}

获取邻域 S_{xy} 的 $\begin{cases} \text{灰度均值 } m_{xy} \\ \text{灰度标准差 } \sigma_{xy} \end{cases}$ 引入非负常数 a, b

构造可变阈值 $T_{xy} = a\sigma_{xy} + bm_{xy}$ 或 $T_{xy} = a\sigma_{xy} + bm_G$

输入图像的阈值化 $g(x, y) = \begin{cases} 1 & f(x, y) > T_{xy} \\ 0 & f(x, y) \leq T_{xy} \end{cases}$

一般方式 $g(x, y) = \begin{cases} 1 & Q(\text{以}(x, y)\text{为中心的局部参数})\text{为真} \\ 0 & Q(\text{以}(x, y)\text{为中心的局部参数})\text{为假} \end{cases}$

一般方式 $g(x, y) = \begin{cases} 1 & Q(\text{以}(x, y)\text{为中心的局部参数})\text{为真} \\ 0 & Q(\text{以}(x, y)\text{为中心的局部参数})\text{为假} \end{cases}$

例: $Q(\sigma_{xy}, m_{xy}) = \begin{cases} \text{真}, & f(x, y) > a\sigma_{xy} \text{ AND } f(x, y) > bm_{xy} \\ \text{假}, & \text{其它情况} \end{cases}$

输入图像

双阈值化(多阈值)

注意: 中等灰度区域未完全分割

局部阈值化

$T_{xy} = a\sigma_{xy} + bm_G$

$a=30$
 $b=1.5$

标准差图像

输入图像的3*3局部

方式3—基于移动平均的可变阈值处理

(1)前提

- 光照不均匀的输入图像
- 感兴趣的目标与图像相比，尺寸很小、或目标过细(如：手写文本图像)

(2)处理方式

- 以zigzag方式，逐行、逐列扫描
- 根据平均笔画的宽度 w ，确定扫描序列长度的经验值 $n=5w$
- 扫描窗口大小: 1行*n列

具体方法: 考虑当前扫描行。

设扫描序列的第 k 步遇到第 k 个新像素，该像素的灰度值 z_k
初始化第 $k(1 \leq k \leq n)$ 个扫描像素所在扫描窗口的灰度均值

$$m(k) = \frac{1}{k} \sum_{j=1}^k z_j$$

第 $k+1$ 步($k \geq n$)处理第 $k+1$ 个像素，扫描窗口的灰度均值：

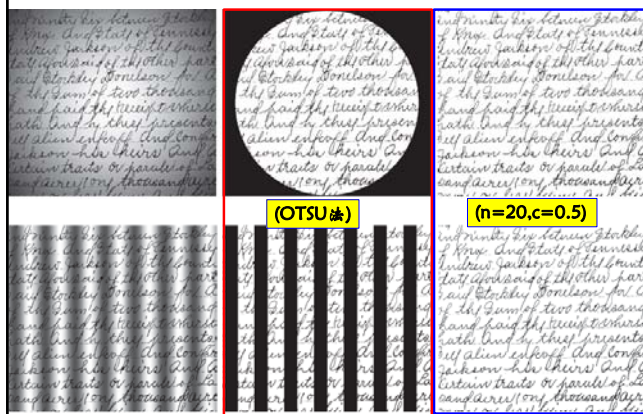
$$\begin{aligned} m(k+1) &= \frac{1}{n} \sum_{i=k+1-(n-1)}^{k+1} z_i = \frac{1}{n} \sum_{i=k+2-n}^{k+1} z_i = \frac{1}{n} \left[z_{k+1} + \sum_{i=k+2-n}^k z_i \right] \\ &= \frac{1}{n} \left[z_{k+1} + \sum_{i=k+1-n}^k z_i - z_{k+1-n} \right] = \frac{1}{n} \sum_{i=k+1-n}^k z_i + \frac{1}{n} [z_{k+1} - z_{k+1-n}] \\ &= m(k) + \frac{1}{n} [z_{k+1} - z_{k+1-n}] \end{aligned}$$

引入常数c,构造可变阈值 $T(k) = c \cdot m(k)$

对于输入图像当前行的第k个像素

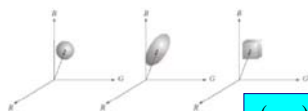
$$\text{阈值化 } g(k) = \begin{cases} 1 & f(k) > T(k) \\ 0 & f(k) \leq T(k) \end{cases}$$

例:基于移动平均的文档图像局部阈值化处理



6. 多通道/彩色图像的阈值化-多变量阈值处理(阈值向量)

例:从给定彩色图像f中,提取具有指定颜色所有区域。



$$\begin{aligned} z(x, y) &= [f_r(x, y) \ f_g(x, y) \ f_b(x, y)]^T \\ a &= [a_r \ a_g \ a_b]^T \text{--感兴趣的平均颜色} \\ D(z, a) &= \|z - a\| = [(z - a)^T (z - a)]^{0.5} \\ \text{或 } D(z, a) &= [(z - a)^T C^{-1} (z - a)]^{0.5} \\ g(x, y) &= \begin{cases} 1 & D(z(x, y), a) < T \\ 0 & \text{其它} \end{cases} \end{aligned}$$

主要内容

10.1 基础知识

10.2 点、线和边缘检测

10.3 阈值处理

10.4 基于区域的分割

区域生长法

分裂合并法

40

图像分割定义

令集合R代表一幅图像占据的整个空间区域,图像分割就是将R划分成满足如下条件的非空子集(子区域) $R_1, R_2, \dots, R_n$ 的过程:

- (1)完备性: 分割必须完全 $\bigcup_{i=1}^n R_i = R$
- (2)连通性: R_i 是一个连通集 $i = 1, 2, \dots, n$
- (3)独立性: $R_i \cap R_j = \emptyset \quad \forall i \neq j$ 各区域不相交
- (4)单一性: $Q(R_i) = \text{TRUE} \quad i = 1, 2, \dots, n$
同一区域内的像素必满足某种一致性属性
- (5)互斥性: $Q(R_i \cup R_j) = \text{FALSE}$ 对于任意邻接区域 R_i, R_j
邻接区域内的像素必不满足某种一致性属性

1. 区域生长法(Region-Growing Method)-自底向上

算法关键:

(1)种子点选取:

选择或确定一组能正确代表目标区域的“种子”像素

(2)应用至输入图像中每个像素的谓词逻辑(Q)

生长准则:

确定生长过程中能够将相邻像素包括进来的准则

生长停止准则:

确定在生长过程中停止的条件或准则。

42

STEP1. 种子点的选取。

根据图像的不同应用，选择一个或一组**种子像素**。

STEP2. 选择一个描述符(生长准则)

STEP3. 区域生长。

(1)从该种子开始向外扩张：

首先把种子像素加入结果集合，然后不断将与集合中各个像素连通(4-连通或8-连通)、且满足**描述符**的像素加入集合。

(2)上述过程不断进行，直到不再有满足条件的新像素加入集合为止。

区域生长算法的主要问题

(1)如何选择或确定一组能正确代表所需区域的种子像素？

具体问题具体分析

--先验知识

军用红外图像目标检测，可选灰度最亮像素作为种子

--无先验知识

可结合图像的灰度直方图选取灰度中像素数目多的像素作为种子

区域生长算法的主要问题

(2)如何确定谓词逻辑Q

满足 $Q \Rightarrow$ 生长过程中将相邻像素合并的准则

具体问题相关(目标和背景的像素分布特点)

--图像数据种类(单色、灰度还是彩色)

--像素间的连通性和邻近性

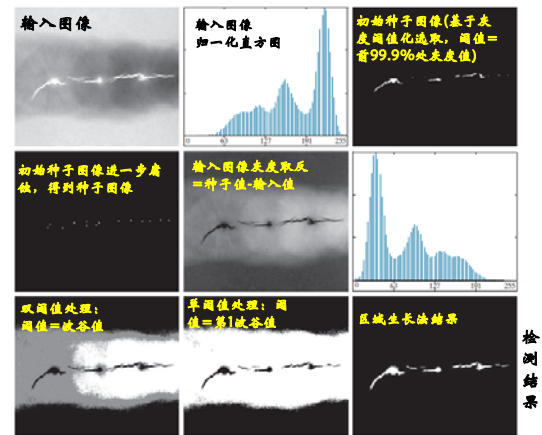
不满足 $Q \Rightarrow$ 生长过程停止的条件或规则

一般是没有满足生长的像素

需考虑图像的局部性质，如灰度、纹理和彩色

或者目标的全局性质，如尺寸、形状等

例：基于区域生长法的X射线图像的缺陷焊缝提取



2. 区域分裂-合并法(Split-Merge Algorithm)——自顶向下

➤ 从全图出发，逐渐分裂至满足条件分割区；同时，每次分裂后，对满足空间邻近性、特性(灰度、颜色、纹理)一致性的区域进行合并。

➤ 实现时，常利用图像数据的**金字塔**或**四叉树**数据结构：

首先，图像分成任意大小且不重叠的初始区域；

然后，根据给定**区域均匀性检验准则**，进一步分裂或合并这些区域，逐步改善区域划分的性能。

直到将图像分成数量最少的均匀区域为止。

➤ 处理的**图像应该为方形**，且图像的**行数、列数为2的整数次幂**

分裂与合并算法描述：

第1步.定义一个划分为区域的初始分割、一致性准则Q、金字塔数据结构；以及四象限区域最小尺寸。

第2步.如果在金字塔数据结构中的**任意一个区域不是一致的(并且大于最小尺寸)**，将其**分裂为四个子区域**；

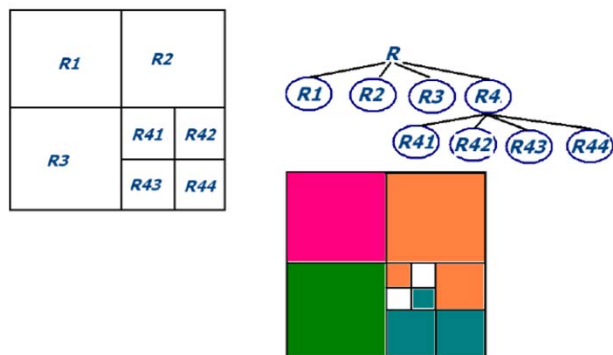
--对于具有相同父节点的任意相邻区域，若合并后满足一致性准则，则**合并**它们

--若没有区域可以合并或分裂，则转置**第3步**

第3步.如果任意两个邻接区域 R_i 和 R_j 可以合并为一个一致性区域(可能来自金字塔的不同层；或者没有相同父节点)，则合并它们。

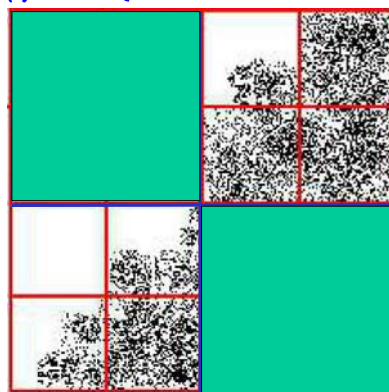
第4步(后处理)如果必须删除小尺寸区域，则将小区域与其最相似的邻接区域合并。

分裂-合并过程（以四叉树结构为例）图示：



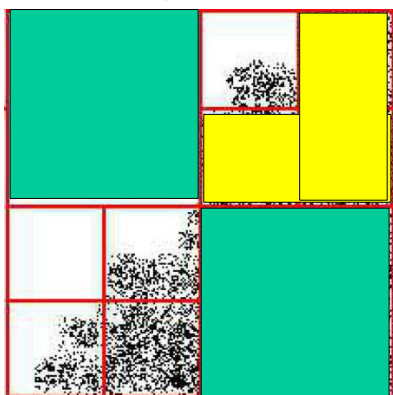
四叉树法灰度图像分割举例：

(1)初始分割：



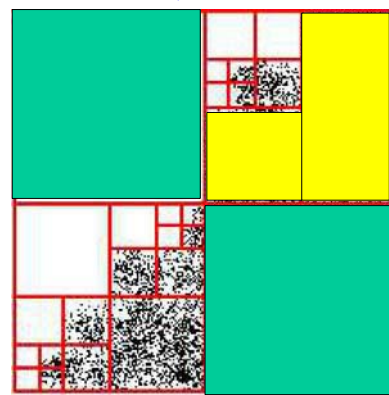
(2)合并处理：

按合并原则对第二层,每四个小块进行合并检测



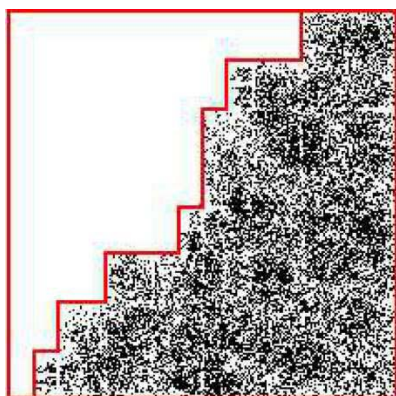
(3)分裂处理

按分裂原则对第二层剩余小块进行分裂检测



(4)后处理

以每块为中心,检查相邻块,符合均匀特性则合并



例：基于分裂-聚合法的灰度图像分割

> 明确分割的目的：

提取围绕致密中心的**环状物质区域**

> 明确不再分裂的规则 Q

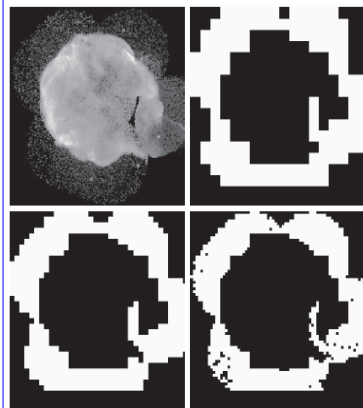
环状区域具有较大的方差,灰度均值介于暗区背景与亮区致密区域

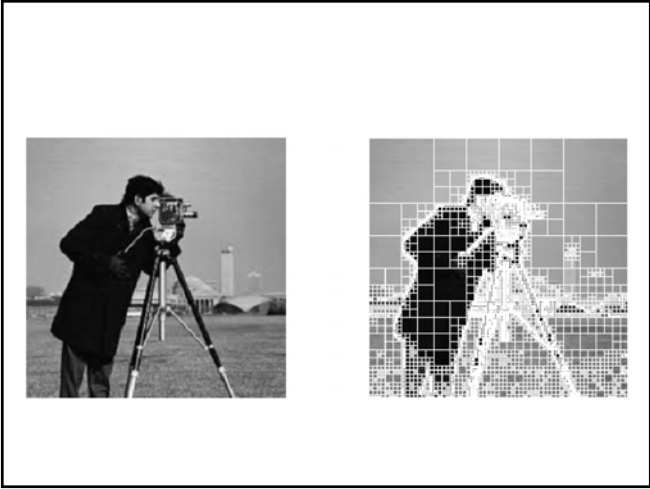
$$Q = \begin{cases} \text{TRUE} & \text{if } \sigma > a \text{ AND } 0 < m < b \\ \text{FALSE} & \text{otherwise} \end{cases}$$

> 明确四象限区域最小尺寸

——分裂终止的条件之一
32*32 16*16 8*8

输入：566*566的X波段图像
输出：二值分割结果





主要内容	
10.1	基础知识
10.2	点、线和边缘检测
10.3	阈值处理
10.4	基于区域的分割
10.5	基于聚类和超像素的区域分割
10.5.1	基于K-均值聚类的区域分割
10.5.2	基于超像素的区域分割

将图像的区域分割视为以像素为样本的聚类。
每个像素样本以颜色、或灰度值为特征进行描述。

给定一幅 M 行 N 列的输入图像 I ，令 $m=M \times N$
该图像中的所有像素构成样本集 $D = \{x_1, \dots, x_Q\}$

对于 $\forall x \in D$ ，有： $x = [x_1 \ \dots \ x_d]^T \in R^d$ 。
若图像 I 为RGB彩色图像，则 $n=3$

基于 K -均值聚类的目的在于将观测图像的各像素划分为 K 个互不相交的像素子集 $C_1, \dots, C_K$ ，它们满足：

$$\{C_1^*, \dots, C_K^*\} = \arg \min_{C_1, \dots, C_K} \left(\sum_{i=1}^k \sum_{x \in C_i} \|x - \mu(C_i)\|^2 \right)$$

输入：样本集 $D = \{x_1, x_2, \dots, x_m\}$;
聚类簇数 k 。

过程：

- 1: 从 D 中随机选择 k 个样本作为初始均值向量 $\{\mu_1, \mu_2, \dots, \mu_k\}$
- 2: repeat
- 3: 令 $C_i = \emptyset \ (1 \leq i \leq k)$
- 4: for $j = 1, 2, \dots, m$ do
- 5: 计算样本 x_j 与各均值向量 $\mu_i \ (1 \leq i \leq k)$ 的距离: $d_{ji} = \|x_j - \mu_i\|_2$;
- 6: 根据距离最近的均值向量确定 x_j 的簇标记: $\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$;
- 7: 将样本 x_j 划入相应的簇: $C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$;
- 8: end for
- 9: for $i = 1, 2, \dots, k$ do
- 10: 计算新均值向量: $\mu'_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$;
- 11: if $\mu'_i \neq \mu_i$ then
- 12: 将当前均值向量 μ_i 更新为 μ'_i
- 13: else
- 14: 保持当前均值向量不变
- 15: end if
- 16: end for
- 17: until 当前均值向量均未更新

输出：簇划分 $C = \{C_1, C_2, \dots, C_k\}$

➢ 初始化方式
➢ 交叉迭代的动态更新方式
➢ 迭代终止条件

或：
 $E(t+1) = \sum_{i=1}^k \|\mu(C_i^{(t+1)}) - \mu(C_i^{(t)})\|^2 \leq T$

基于K-均值聚类的灰度图像区域分割

左：原始输入图像688*688 右：区域分割结果(K=3)

主要内容	
10.1	基础知识
10.2	点、线和边缘检测
10.3	阈值处理
10.4	基于区域的分割
10.5	基于聚类和超像素的区域分割
10.5.1	基于K-均值聚类的区域分割
10.5.2	基于超像素的区域分割

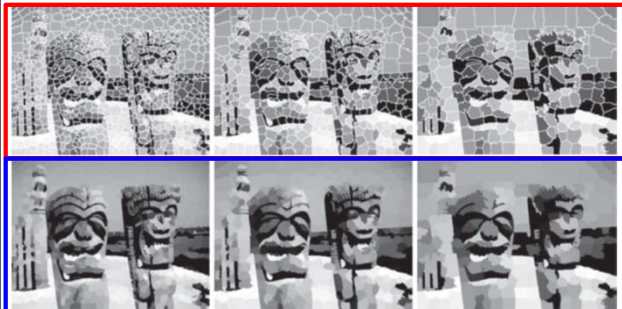
----基于简单线性迭代聚类(SLIC,2012)的超像素分割

SLIC—Simple Linear Iterative Clustering

木雕图像(600行*800列)的超像素及其超像素图像

超像素数目:

左, 1000; 中, 500; 右, 250



A. 什么是超像素(super-pixel)

基于像素的空间位置、颜色、纹理、亮度等底层特征的一致性, 由若干空间位置相邻像素组成的有一定视觉意义的不规则像素块、或像素集合。

以少数的超像素代替图像的大规模像素来表达图像特征,
---关于图像划分的一种具有空间压缩意义的表达。

超像素法区域分割通常作为下游任务的预处理环节

---图像分割

---目标检测与识别

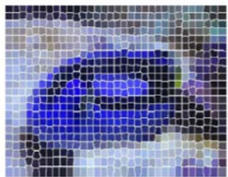
彩色图像及其超像素块



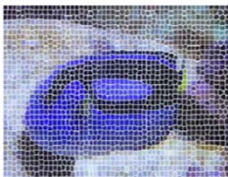
(A) Image



(C) 280 pixels, 4 Proximity



(B) 660 pixels, 10 Proximity



(D) 1750 pixels, 4 Proximity

B. SLIC 算法描述

给定一幅由N个像素组成的图像I, 图像中所有像素Z构成样本集合 $D = \{z_i, i = 1, \dots, N\}$

设图像I中生成的大小近似相等的超像素数目为 n_{sp}

则图像中每个超像素的边长(或相邻超像素的网格间距)为

$$s = \left[\text{round} \left(N / n_{sp} \right) \right]^{1/2}$$

对于 $\forall z \in D$, 以像素颜色特征及空间坐标描述该样本:

$$z = [r, g, b, x, y]^T$$

或

$$z = [l, \alpha, \beta, r, g, b, x, y]^T$$

B. SLIC 算法描述(续)

设置SLIC算法终止条件(非负阈值T), SLIC具体实现步骤:

STEP1.超像素中心的初始化

(1)在图像坐标, 以水平/垂直规则步长S进行等间隔采样, 得到 n_{sp} 个初始数目的超像素聚类中心:

$$m_i = [r_i, g_i, b_i, x_i, y_i]^T, i = 1, 2, \dots, n_{sp}$$

(2) n_{sp} 个聚类中心的位置扰动。

为避免超像素中心的初始化位置落在输入图像的边缘点或噪声点处, 分别考察上述每个中心初始位置的 3×3 邻域, 计算邻域内各像素的梯度幅值, 将超像素中心的初始位置移至其 3×3 邻域内梯度幅值局部最小值点处。

图像I中任意位置(x,y)处像素p的梯度幅值:

$$G(x, y) = [\|I(x, y) - I(x-1, y)\|_2^2 + \|I(x, y) - I(x, y-1)\|_2^2]^{1/2}$$

B. SLIC 算法描述(续)—接前页

STEP2.各像素样本信息的初始化

对于图像中的每个像素位置p:

➢ 设置该像素的标签信息为 $L(p) = -1$

➢ 设置该像素关于超像素中心的最近距离为 $d(p) = \infty$

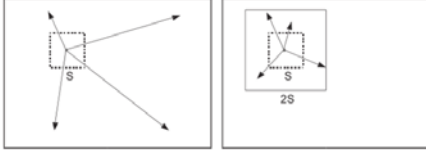
注意: 此处的距离由空间距离和颜色距离构成。

STEP3. 按照就近距离法，为每个超像素中心分配样本。

➤ 对于每个超像素中心 $m_i = [r_i, g_i, b_i, x_i, y_i]^T, i = 1, 2, \dots, n_{sp}$ 计算 m_i 的 $2s \times 2s$ 邻域内每个像素 p 对应的样本 z 与 m_i 的距离：

$$D_i(p) = \left\{ [d_{color}(z, m_i)/255]^2 + c^2 [d_{spatial}(z, m_i)/s]^2 \right\}^{1/2}$$

参数 c (称之为 compactness, 紧密度) 用于平衡两种距离的相对重要性: c 值越大, 表明超像素块的生成更看重像素之间的空间邻近性, 得到的超像素块越规则。



左, 标准的K-均值聚类搜索整幅图像 右, SLIC搜索其 $2s \times 2s$ 邻域

STEP3. 按照就近距离法，为当前的每个超像素中心分配样本。

$$D_i(p) = \left\{ [d_{color}(z, m_i)]^2 + c^2 [d_{spatial}(z, m_i)/s]^2 \right\}^{1/2}$$

其中:

$$d_{color}(z, m_i) = [(r - r_i)^2 + (g - g_i)^2 + (b - b_i)^2]^{1/2}$$

$$d_{spatial}(z, m_i) = [(x - x_i)^2 + (y - y_i)^2]^{1/2}$$

➤ 若 $D_i(p) < d(p)$, 则更新像素 p 的信息: $\begin{cases} d(p) = D_i(p) \\ L(p) = i \end{cases}$

STEP4. 更新 n_{sp} 个超像素 $C_1, \dots, C_{n_{sp}}$ 的中心:

对于第 i 个超像素(即: 第 i 簇) C_i , 其中心更新为:

$$m_i = \frac{1}{|C_i|} \sum_{z \in C_i} z = \frac{1}{\sum_{z \in D} I(L(P(z))=i)} \sum_{z \in D} z \cdot I(L(P(z))=i)$$

STEP5. 收敛性检验。

计算更新前后, “ n_{sp} ” 超像素块中心距离的累加和 E 。

若 $E < T$, 则迭代终止, 进入STEP6;

否则, 重复STEP2~STEP5。

STEP6. 超像素区域的后处理。

(1) 超像素的连通区域增强

为有效避免前述步骤可能导致的少量具有相同簇标签的像素空间不连通的情况, 以及避免少数超像素包含数量过少的像素, 引入该环节。将空间不连通的过小超像素块中各像素重新赋以空间邻域中具有更多数量像素的超像素标签。

(2) 以超像素块内各像素的平均颜色替换原始图像在该超像素块内的各像素颜色, 得到超像素图像并输出。

(3) 输出各超像素块的中心向量:

$$m_i = [r_i, g_i, b_i, x_i, y_i]^T, i = 1, 2, \dots, n_{sp}$$

B. SLIC 算法描述

给定一幅由 N 个像素组成的图像 I , 图像中所有像素 z 构成样本集合 $D = \{z_i, i = 1, \dots, N\}$

设图像 I 中生成的大小近似相等的超像素数目为 n_{sp}

则图像中每个超像素的边长(或相邻超像素的网格间距)为

$$s = \left\lceil \text{round} \left(\frac{N}{n_{sp}} \right) \right\rceil^{1/2}$$

对于 $\forall z \in D$, 以像素颜色特征及空间坐标描述该样本:

$$z = [r, g, b, x, y]^T$$

或

$$z = [l, \alpha, \beta, r, g, b, x, y]^T$$

SLIC算法的“超像素中心向量的初始化”

在图像坐标系等间隔采样, 得到各超像素的初始中心

```
# Initial regions: 为每个超像素记录其所属的超像素编号(序号, 符号)
regionsID = createImage2I(width, height)
for x,y in itertools.product(range(0, regW), range(0, regH)):
    ry, rx = y * regionSide, x * regionSide
    # Position
    py, px = int(ry + halfRegionSide), int(rx + halfRegionSide)
    regionPos[y,x] = [py, px]
    minGradient = FLT_MAX
    for wx,wy in itertools.product(range(px-1, px+2), range(py-1, py+2)):
        if (wx < height and wx < width):
            if gradient[wy,wx] < minGradient:
                minGradient = gradient[wy,wx]
                regionPos[y,x] = [wy, wx]
    # Colour
    colour = [0, 0, 0]
    npts = 0.0
    for wx,wy in itertools.product(range(rx, rx + regionSide), range(ry, ry + regionSide)):
        if wy>=0 and wy<height and wx>=0 and wx<width:
            regionsID[wy,wx] = [y, x]
            colour += inputImage[wy,wx]
            npts += 1
    if npts > 0:
        regionColour[y,x] = [int(colour[0] / npts), int(colour[1] / npts), int(colour[2] / npts)]
```


SUC 中各超像素块的信息更新

```
# Modify regions
for itr in range(0, numIter):

    # Values for new regions
    newRegionColour = createImageF(regW, regH,3)
    newRegionPos = createImageUV(regW, regH)
    newRegionSize = createImageF(regW, regH)

    # Per pixel
    for x,y in itertools.product(range(0, width), range(0, height)):
        ry, rx = regionsID[y,x]
        colour = [float(inputImage[y,x][0]), float(inputImage[y,x][1]), \
                  float(inputImage[y,x][2])]
        minD = [FLT_MAX, ry, rx]

        for wx,wy in itertools.product(range(rx-2, rx+2), range(ry-2, ry+2)):
            if wy>0 and wy<regH and wx>0 and wx<regW:
                ds = sqrt((regionPos[wy,wx][0]- y)**2 + (regionPos[wy,wx][1]- x)**2)
                dc = sqrt((float(regionColour[wy,wx][0]) - colour[0])**2 + \
                          (float(regionColour[wy,wx][1]) - colour[1])**2 + \
                          (float(regionColour[wy,wx][2]) - colour[2])**2)
                D = dc/255.0 + (m / regionSize) * ds
                if D < minD[0]:
                    minD = [D, wy, wx]

        [, minY, minX] = minD
        newRegionColour[minY, minX] += colour
        newRegionPos[minY, minX] += [y,x]
        newRegionSize[minY, minX] += 1
        regionsID[y, x] = [minY, minX]

    # Update regions
    for x,y in itertools.product(range(0, regW), range(0, regH)):
        if newRegionSize[y,x] > 0:
            regionPos[y,x] = newRegionPos[y,x] / newRegionSize[y,x]
            regionColour[y,x] = newRegionColour[y,x] / newRegionSize[y,x]
```

为更新后的超像素分配空间：
颜色，位置，该簇的像素数目

获取更新前，当前像素的超像素标签、该像素的颜色信息、与超像素中心的最近距离

计算当前像素与5*5邻域超像素中心的距离，超像素标签、该像素的颜色信息、与超像素中心的最近距离，以确定该像素的最近超像素

将与该像素距离最近超像素块内记录：该像素的颜色信息、空间位置、该簇的像素数目更新该像素所属的簇标签

更新各个超像素块的中心位置：平均位置，平均颜色

/* Initialization */

- Initialize cluster centers $C_k = [l_k, a_k, b_k, x_k, y_k]^T$ by sampling pixels at regular grid steps S .
- Move cluster centers to the lowest gradient position in a 3×3 neighborhood.
- Set label $l(i) = -1$ for each pixel i .
- Set distance $d(i) = \infty$ for each pixel i .

/* Assignment */

repeat

for each cluster center C_k do

for each pixel i in a $2S \times 2S$ region around C_k do

Compute the distance D between C_k and i .

if $D < d(i)$ then

set $d(i) = D$

set $l(i) = k$

end if

end for

end for

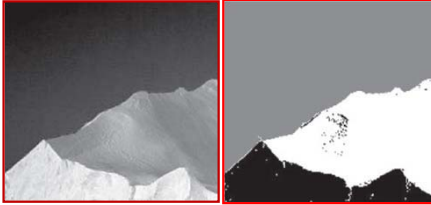
/* Update */

Compute new cluster centers. Compute residual error E .

until $E \leq \text{threshold}$

74

图像的K-均值聚类



图像的超像素分割+K均值聚类

